

컴퓨터 구조론

HW1 Report



32217072 모바일시스템공학과 김도익
컴퓨터 구조론 HW1

목차

1. 서론

2. 본론

- 1) 배경지식
- 2) 디자인
- 3) 코드설명
- 4) 예외 처리
- 5) 구현
- 6) 프로그램 실행

3. 결론

1. 서론

폰 노이만의 컴퓨터 구조는 현대 컴퓨터 구조의 기반이 되는 중요한 개념 중 하나이다. 이 구조는 프로그램과 데이터를 메모리에 저장하고, 이를 연산하기 위해 CPU에서 실행하는 방식으로 작동한다. 본 레포트에서는 컴퓨터 구조의 기본 원리를 심도 있게 탐구하기 위해, 어셈블리어에서 사용되는 여러 가지 명령어와 레지스터를 구현하고 실행하는 과정을 C 언어로 모델링하여 실제 CPU에서 명령어가 어떻게 실행되는지에 대한 이해를 높이고 컴퓨터구조를 깊이 이해하고자 한다.

2. 본론

1) 배경지식

● 레지스터종류

s레지스터: s0부터 s7까지의 8개 레지스터이다. 이들은 레지스터의 값을 저장하는데 사용한다.

t레지스터: t0부터 t9까지의 레지스터이다. 이들은 임시 데이터 저장 및 연산을 위해 사용된다.

v레지스터: v0 와 v1레지스터는 함수 호출 후 함수의 결과 값을 저장하는데 사용된다. 본 코드에서는 v0만을 사용하여 return 값으로 사용했다.

Zero레지스터: 항상 0을 갖는 레지스터이다.

- 연산자종류

ADD rd rs rt: rs의 값과 rt레지스터의 값을 더해서 rd에 저장한다.

SUB rd rs rt: rs의 값에서 rt를 뺀 값을 rd에 저장한다.

MUL rd rs rt: rs의 값과 rt의 값을 곱하여 rd에 저장한다.

DIV rd rs rt: rs의 값에서 rt를 나눈 값을 rd에 저장한다.

NOP: 어떤 작업도 하지 않는다.

JMP target_address: pc(program counter)를 target_address로 변경한다.

BEQ rs rt offset: rs와 rt에 저장된 값이 서로 같으면 프로그램은 offset만큼의 주소로 이동한다. 그렇지 않으면 다음 명령어로 진행된다.

BNE rs rt offset: rs와 rt에 저장된 값이 서로 다르면 프로그램은 offset만큼의 주소로 이동한다. 그렇지 않으면 다음 명령어로 진행된다.

SLT rd rs rt: rs와 rt에 저장된 값중 rs값이 rt값보다 작으면 rd레지스터에 1을 저장하고 그렇지 않으면 0을 저장한다.

2) 디자인

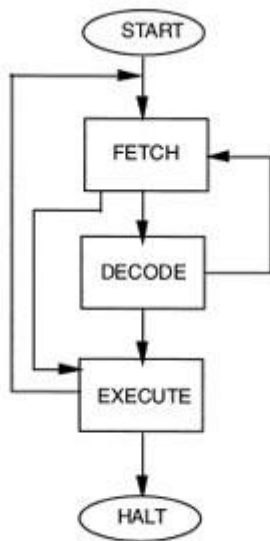


FIGURE 6 Basic instruction processing cycle.

코드의 구조는 main 함수에서 시작해, 파일 읽기, 명령어 분석 (decode), 계산(execute)함수로 이어진다. 시뮬레이터는 파일을 한 줄씩 읽어 메모리에 저장한후 명령어 주기에 따라 각각을 처리한다.

메모리 구조는 2차원 문자 배열로 표현되며, 명령어 레지스터(IR), 프로그램 카운터(PC), 레지스터(s, t, v, zero)로 구성된다.

Fetch: PC에 해당하는 메모리 위치에서 명령어 한줄을 IR배열로 가져온다. 가져온 명령어는 Decode단계로 전달된다.

Decode: 가져온 명령어는 decode함수에서 분석된다. 명령어는 opcode, operand1, operand2, operand3으로 분해되어, 실행할 연산의 종류와 사용될 데이터 또는 데이터 위치를 식별한다.

Execute: Decode된 명령어는 execute함수를 통해 처리된다. 이 함수는 사칙연산, NOP, JMP, BEQ, BNE, SLT 의 연산을 구분하여 각각의 연산에 맞는 함수를 호출한다.

Halt: 모든 명령어의 실행이 끝나면, 종료된다.

3) 코드설명

t,s,v,zero레지스터: 구조체 배열로 t의 크기는 10, s의 크기는 8, v의 크기는 1을 할당하여 정의하였다. S0지스터의 경우 s구조체 배열의 0번 index에, t2의 경우 t구조체 배열의 2번 index에 저장하였고 zero는 #define 전처리를 사용해 0으로 처리하였다.

Decoded_instruction: inst라는 구조체로 정의하였다. 하나의 명령어를 나타내며 각 명령어는 opcode와 operand1~operand3으로 구성된다.

memory: 파일을 load할 100 X 100 크기의 char형 2차원 배열을 생성하여 가상의 메모리를 구현하였다.

IR: 크기가 char형100인 IR배열을 생성하여 memory의 한 행을 fetch한다.

pc: 명령어가 있는 행을 지정하는 pc를 정수형 변수로 설정하여 올바른 위치에 있는 명령어를 수행하게끔 구현하였다.

decode 함수: 문자열 형태의 명령어를 받아서 각각의 구성요소로 분리해 구조체에 저장한다. 추가적인 토큰이 더 이상 없는지 검사하여, 만약 있으면 오류메시지를 출력하고 함수를 종료한다.

execute 함수: 디코딩된 명령어를 실제로 실행하는 역할을 한다. opcode를 검사하여 해당하는 연산을 결정한다.

4) 예외처리

1. 각 토큰의 길이가 최대 허용 길이를 초과하는지 검사

Opcode, operand1, operand2, operand3에 대해 모두 확인한다.

2. 토큰이 없거나 5개 이상인 경우 오류 처리

Ex) ADD s0 t0 t1 t2

3. 적절치 않은 opcode 오류 처리

정의되지 않은 명령어가 실행될 경우 오류로 처리된다.

4. execute_LW, execute_ADD, execute_SUB, execute_MUL, execute_DIV 함수에서 레지스터 유형의 유효성을 검사하고 index의 유효성을 검사

s의 index는 8이상인 경우, t의 index는 10이상인 경우, v의 index는 1인 경우 오류처리 한다

레지스터자리에 immediate값이 오는 경우 오류처리 한다.

zero레지스터가 operand1 자리에 오는 경우 오류처리 한다.

Ex) LW zero 0x2, ADD s9 t10 t22, ADD v1 s0 t0, SUB s0 10 30

5. execute_JMP 함수에서 JMP함수가 레지스터를 사용하려고 하는 경우를 검사

Ex) JMP s0, JMP t0

6. execute_DIV 함수에서 0으로 나누는 경우를 오류처리

Ex) DIV s0 t0 zero, DIV s0 t0 t1 (t1에 저장된 값이 0일때)

.

5) 구현

Implemented parts	o/x
ADD, SUB, MUL, DIV, NOP	0
JMP	0
BEQ	0
BNE	0
SLT	0

프로그램이 testcase에서 정상적으로 실행되고 명령어가 올바르게 처리되었지만, JMP, BEQ, BNE 같은 분기 명령어에서 한 가지 중요한 이슈를 확인했다. 특히 이미 실행된 분기 명령어가 반복해서 실행될 때 이 명령어를 스킵하는 로직을 구현하는 데 어려움을 겪었다. 이 문제는 분기 명령어가 실행될 때마다 프로그램의 실행 흐름이 변경되어야 함에도 불구하고 실행된 코드를 다시 실행하며 발생하는 무한루프의 원인이 되었다.

6) 프로그램 실행

KOI에서 프로그램실행 방법

- ① 터미널을 열고 `gcc -o 32217072 32217072.c` 커맨드를 작성하여 컴파일 한다.
- ② txt파일을 준비한다. 파일이름은 test로 가정한다.
- ③ `./32217072 test.txt` 커맨드를 작성한다.

KOI에서 프로그램 실행 출력

testcase는 강의자료 5 ~ 9페이지 example을 순서대로 사용
(txt 파일에 r레지스터가 있다면 v레지스터로 변경)

1.testcase1.txt

```
LW s0 0xF
LW s1 0x4
ADD v0 s0 s1
```

```
koicloud@ca-32217072:~/vscode$ ./32217072 testcase1.txt
(line 1) LW s0 0xF
32217072> Loaded 15 to s0
(line 2) LW s1 0x4
32217072> Loaded 4 to s1
(line 3) ADD v0 s0 s1
32217072> Added s0(15) to s1(4) and changed v0 to 19
koicloud@ca-32217072:~/vscode$
```

2.testcase2.txt

```
LW s0 0x2
LW s1 0x4
MUL v0 s0 s1
```

```
koicloud@ca-32217072:~/vscode$ ./32217072 testcase2.txt
(line 1) LW s0 0x2
32217072> Loaded 2 to s0
(line 2) LW s1 0x4
32217072> Loaded 4 to s1
(line 3) MUL v0 s0 s1
32217072> Multiplied s0(2) to s1(4) and changed v0 to 8
koicloud@ca-32217072:~/vscode$
```

3.testcase3.txt

```
LW s0 0x1
LW s1 0x2
LW s2 0x3
LW s3 0x4
ADD t0 s0 s1
ADD t1 s2 s3
SUB v0 t0 t1
```

```
koicloud@ca-32217072:~/vscode$ ./32217072 testcase3.txt
(line 1) LW s0 0x1
32217072> Loaded 1 to s0
(line 2) LW s1 0x2
32217072> Loaded 2 to s1
(line 3) LW s2 0x3
32217072> Loaded 3 to s2
(line 4) LW s3 0x4
32217072> Loaded 4 to s3
(line 5) ADD t0 s0 s1
32217072> Added s0(1) to s1(2) and changed t0 to 3
(line 6) ADD t1 s2 s3
32217072> Added s2(3) to s3(4) and changed t1 to 7
(line 7) SUB v0 t0 t1
32217072> Subtracted t0(3) to t1(7) and changed v0 to -4
koicloud@ca-32217072:~/vscode$
```

4.testcase4.txt

```
LW s0 0x5
LW s1 0xA
ADD t0 s0 s1
JMP 0x8
LW s0 0x1
LW s1 0x2
ADD t0 s0 s1
LW s2 0x3
MUL v0 t0 s2
```

```
koicloud@ca-32217072:~/vscode$ ./32217072 testcase4.txt
(line 1) LW s0 0x5
32217072> Loaded 5 to s0
(line 2) LW s1 0xA
32217072> Loaded 10 to s1
(line 3) ADD t0 s0 s1
32217072> Added s0(5) to s1(10) and changed t0 to 15
(line 4) JMP 0x8
32217072> Jumped to line 8
(line 8) LW s2 0x3
32217072> Loaded 3 to s2
(line 9) MUL v0 t0 s2
32217072> Multiplied t0(15) to s2(3) and changed v0 to 45
koicloud@ca-32217072:~/vscode$
```

5.testcase5.txt

```
LW s0 0x5
LW s1 0xA
BEQ s0 s1 0x6
ADD v0 s0 zero
JMP 0x7
ADD v0 s1 zero
```

```
koicloud@ca-32217072:~/vscode$ ./32217072 testcase5.txt
(line 1) LW s0 0x5
32217072> Loaded 5 to s0
(line 2) LW s1 0xA
32217072> Loaded 10 to s1
(line 3) BEQ s0 s1 0x6
32217072> checked if s0(5) is equal to s1(10) and didn't jump to line 6
(line 4) ADD v0 s0 zero
32217072> Added s0(5) to z0(0) and changed v0 to 5
(line 5) JMP 0x7
32217072> Jumped to line 7
(line 7) NOP
32217072> No Operation
koicloud@ca-32217072:~/vscode$
```

NOP

6.testcase6.txt

LW s0 0x5

LW s1 0xA

BNE s0 s1 0x6

ADD v0 s0 zero

JMP 0x7

ADD v0 s1 zero

NOP

```
koicloud@ca-32217072:~/vscode$ ./32217072 testcase6.txt
(line 1) LW s0 0x5
32217072> Loaded 5 to s0
(line 2) LW s1 0xA
32217072> Loaded 10 to s1
(line 3) BNE s0 s1 0x6
32217072> checked if s0(5) is not equal to s1(10) and jumped to line 6
(line 6) ADD v0 s1 zero
32217072> Added s1(10) to z0(0) and changed v0 to 10
(line 7) NOP
32217072> No Operation
koicloud@ca-32217072:~/vscode$
```

7.testcase7.txt

LW s0 0x5

LW s1 0xA

SLT t0 s0 s1

BNE t0 zero 0x7

ADD v0 s0 zero

JMP 0x8

ADD v0 s1 zero

NOP

```
koicloud@ca-32217072:~/vscode$ ./32217072 testcase7.txt
(line 1) LW s0 0x5
32217072> Loaded 5 to s0
(line 2) LW s1 0xA
32217072> Loaded 10 to s1
(line 3) SLT t0 s0 s1
32217072> checked if s0(5) is less than s1(10) and set 1 to t0
(line 4) BNE t0 zero 0x7
32217072> checked if t0(1) is not equal to z0(0) and jumped to line 7
(line 7) ADD v0 s1 zero
32217072> Added s1(10) to z0(0) and changed v0 to 10
(line 8) NOP
32217072> No Operation
koicloud@ca-32217072:~/vscode$
```

8.testcase8.txt

LW s0 0x2

LW s1 0x10

ADD v0 s0 s1

NOP

```
koicloud@ca-32217072:~/vscode$ ./32217072 testcase8.txt
(line 1) LW s0 0x2
32217072> Loaded 2 to s0
(line 2) LW s1 0x10
32217072> Loaded 16 to s1
(line 3) ADD v0 s0 s1
32217072> Added s0(2) to s1(16) and changed v0 to 18
(line 4) NOP
32217072> No Operation
koicloud@ca-32217072:~/vscode$
```

3. 결론

기본적인 컴퓨터 구조의 명령어 실행 메커니즘을 시뮬레이션하는 프로그램을 구현함으로써 프로세서, 메모리, 레지스터 및 명령어 사이클을 모델링하여, 이론적 학습을 실제와 연결하는데 중점을 두었다.

이 프로젝트를 진행하며 각 명령어마다 다른 operand 개수와 다른 레지스터를 분류하고 처리하는 과정이 어려웠고 구현하기 힘들었다. 또 수 많은 예외를 찾고 처리를 구현하는 것이 가장 어려웠다.

아쉬운 점은 분기명령어가 두 번 실행되면 스킵하고 다음 명령어를 실행하는 로직을 구현하지 못한 것이다. 결과적으로 이런 이슈는 프로그램의 분기 명령어 처리 방식을 개선하고, 보다 정교한 메커니즘을 통해 해결될 수 있다. 이 경험을 통해 프로그램의 분기 및 흐름 제어 기능의 중요성을 알 수 있었고, 이를 개선하기 위한 구체적인 해결 방안을 모색하는 기회로 활용될 것이다. 또한 코드의 가독성과 유지 보수를 향상시키기 위해 함수의 분리 및 모듈화를 진행시켜야 할 것임을 느꼈다.